

# OpenSeek-8 技术报告

## 1. 任务概述

本任务面向 OpenSeek-8 kernel generation 场景，目标是根据每条测试样例中的自然语言功能描述、Wrapper Entry Information、参数说明和数学定义，自动生成一段可执行的 Python 代码作为预测结果。与普通算法题不同，该任务的输出不是一个数值、字符串或分类标签，而是完整的 Python 函数实现。最终提交文件采用 JSONL 格式，每一行对应一道测试题，字段中保存测试样例 ID 和生成的代码字符串。

本方案的核心思想是：不要让模型在每道题上都从零开始理解题目、推断函数名、选择算子、处理参数并编写代码，而是先离线构建一组中间知识文件，再在最终生成阶段把这些知识文件作为 Prompt-KB 提供给 Qwen 模型。这样可以把任务拆成两个阶段：

1. **中间知识构建阶段**：根据训练样例、测试题描述和高频算子模式，生成算子手册与逐题分析文件；
2. **最终代码生成阶段**：主脚本读取中间文件，针对每个测试样例构造短而精准的 Prompt，调用 Qwen 生成代码，并进行清洗、校验、修复和规则回退。

本报告对应的主代码文件为：

```
1 | task8_prompt_kb_generator copy.py
```

额外构建的中间文件生成脚本为：

```
1 | build_task8_kb_files_no_autoscore.py
```

最终使用的三个中间知识文件为：

```
1 | task8_operator_manual.md
2 | task8_per_question_analysis.jsonl
3 | task8_per_question_analysis.md
```

其中，`task8_operator_manual.md` 和 `task8_per_question_analysis.jsonl` 会被主预测脚本直接读取；`task8_per_question_analysis.md` 是逐题分析的 Markdown 可读版本，主要用于人工检查、报告展示和版本复核，不直接参与主脚本推理。

## 2. 文件组成与作用

### 2.1 主预测脚本

主预测脚本是整个方案的最终执行入口，负责读取原始任务数据、读取中间知识文件、调用 Qwen 生成代码、验证代码格式，并输出 JSONL 提交文件。

```
1 | task8_prompt_kb_generator copy.py
```

脚本中的核心路径配置如下：

```
1 FILE_PATH = "/Users/ks/Desktop/LongContext-ICL-Annotation/data/openssl-  
8_kernel_generation.json"  
2 OUT_PATH = "/Users/ks/Desktop/LongContext-ICL-  
Annotation/outputs/experiment/task8_prompt_kb_fixed_v4_submit.jsonl"  
3  
4 OPERATOR_MANUAL_PATH = "/Users/ks/Desktop/LongContext-ICL-  
Annotation/task8_operator_manual.md"  
5 PER_QUESTION_ANALYSIS_PATH = "/Users/ks/Desktop/LongContext-ICL-  
Annotation/task8_per_question_analysis.jsonl"  
6  
7 MODEL_NAME = "/Qwen3-4B/Qwen/Qwen3-4B"  
8 MAX_WORKERS = 8  
9  
10 USE_MODEL = True  
11 ENABLE_REPAIR = True  
12 ENABLE_RULE_FALLBACK = True
```

上述配置说明：

- `FILE_PATH` 是原始任务数据文件路径；
- `OUT_PATH` 是最终提交 JSONL 输出路径；
- `OPERATOR_MANUAL_PATH` 指向全局算子手册；
- `PER_QUESTION_ANALYSIS_PATH` 指向逐题结构化分析 JSONL；
- `MODEL_NAME` 指定 Qwen 模型；
- `MAX_WORKERS` 用于多线程并发生成；
- `USE_MODEL` 控制是否调用模型；
- `ENABLE_REPAIR` 控制模型输出失败后是否尝试修复；
- `ENABLE_RULE_FALLBACK` 控制是否启用规则回退代码生成。

## 2.2 中间文件一： `task8_operator_manual.md`

`task8_operator_manual.md` 是任务级算子使用手册。它根据题目输入和参考答案中出现的 PyTorch/F API 进行整理，每个高频算子通常包含以下信息：

- 算子名称；
- 用途说明；
- 典型使用场景；
- 简短 demo 代码。

例如文件中包含如下类型的内容：

```

1  ## `F.linear`
2
3  - **用途**: 线性层  $y = xA^T + b$ 。
4  - **典型场景**: linear+activation 复合算子。
5  - **Demo**:
6
7  ```python
8  import torch
9  import torch.nn.functional as F
10 y = F.linear(input, weight, bias)

```

1  
2 该文件的作用是为最终生成脚本提供全局算子知识。主脚本不会每次都把完整手册塞进 Prompt，而是根据当前题目的 family 选择相关片段。例如：

```

3
4  - `conv_norm_pool` 类题目优先选择 `F.conv2d`、`F.batch_norm`、pooling 等内容；
5  - `matmul_linear` 类题目优先选择 `F.linear`、`torch.bmm`、`torch.matmul`、`torch.mm`、`torch.mv` 等内容；
6  - `linalg` 类题目优先选择 `torch.linalg.solve`、`torch.linalg.svd`、`torch.linalg.qr`、`torch.linalg.cholesky` 等内容；
7  - `activation` 和 `elementwise_math` 类题目优先选择 `F.relu`、`F.gelu`、`torch.sqrt`、`torch.exp`、`torch.log` 等内容。

```

8  
9 这样做可以降低 Prompt 长度，同时让模型在生成当前题代码时看到最相关的算子规则。

```

10
11 ### 2.3 中间文件二: `task8_per_question_analysis.jsonl`
12
13 `task8_per_question_analysis.jsonl` 是逐题结构化分析文件，每行对应一道测试样例。该文件记录每道题的函数名、任务类型、功能描述、数学定义、算子链、实现策略和公开简化解题步骤。

```

14  
15 典型字段设计如下：

```

16
17 ```json
18 {
19   "id": "openseek-8-501f776ba20444458ac14dd7292cc913",
20   "function": "fused_bmm_rmsnorm_gelu_dropout_sub",
21   "family": "matmul_linear",
22   "description": "Performs a fused operation combining batch matrix multiplication, RMS normalization, GELU activation, dropout, and subtraction.",
23   "wrapper_signature": "fused_bmm_rmsnorm_gelu_dropout_sub(input1, input2, other, normalized_shape, dropout_p=0.5, training=True, approximate='none', eps=1e-5, *, out=None) -> Tensor",
24   "math": "Z=bmm(X,Y); Z_norm=RMSNorm(Z); G=GELU(Z_norm); D=Dropout(G); output=D-O",
25   "other": "other must be broadcastable to output shape",
26   "detected_ops": ["torch.bmm", "custom_rms_norm", "F.gelu", "F.dropout"],
27   "answer_apis": ["torch.bmm", "torch.rsqrt", "torch.mean", "F.gelu", "F.dropout"],
28   "answer_summary": "Use PyTorch fallback implementation with _write_out support.",
29   "thinking_steps": [
30     "先识别 wrapper 函数名并保持同名定义。",
31     "根据功能描述将任务拆成 bmm、RMSNorm、GELU、Dropout、Sub 五个步骤。",
32     "优先使用 PyTorch API 保证语义正确，避免复杂 Triton。",

```

```
33     "处理 out= 参数并返回写回后的结果。"
34 ]
35 }
```

这里的 `thinking_steps` 不是隐藏思维链，也不是要求模型输出完整内部推理过程，而是公开可审计的简化步骤摘要。它的作用类似工程执行清单，用于提醒最终生成模型按照哪些步骤实现代码。

该 JSONL 文件会被主脚本通过 `sample id` 检索。最终生成 Prompt 时，每道题只会插入当前题对应的分析内容，而不是把 166 道题全部塞入上下文。

## 2.4 中间文件三： `task8_per_question_analysis.md`

`task8_per_question_analysis.md` 是 JSONL 文件的可读版本。它不是最终预测脚本必须读取的输入文件，但在工程上有重要意义：

1. 方便人工检查每道题的 wrapper、family、算子链是否合理；
2. 方便技术报告展示逐题拆解过程；
3. 方便在不同候选版本之间比较分析质量；
4. 方便发现 JSONL 中字段缺失、题目顺序异常或 family 分类错误。

该文件通常包含共性总结，例如：

```
1 总题数：166。
2 任务共同模式：自然语言功能描述 + Wrapper Entry Information + 参数/数学定义 → 生成同名
  Python/Triton wrapper。
3 高稳策略：先保证函数名、import、参数兼容、out/inplace 支持，再用 PyTorch API 实现语义；复杂
  Triton 仅在必要且简单时使用。
4 family 分布：linalg=58, matmul_linear=45, reduction=27, conv_norm_pool=24,
  attention_softmax_loss=6, indexing=4, activation=2
```

该分布说明，Task8 中线性代数、矩阵乘、归约和卷积归一化类任务占比较高。如果强行生成复杂 Triton kernel，容易出现维度、stride、mask、block size 或参数签名错误。因此本方案把“优先 PyTorch fallback”作为核心策略。

## 3. 总体流程设计

整体流程可以分为两个阶段：

```
1 阶段一：中间知识文件构建
2 原始任务 JSON
3   ↓
4 构造一次大于 20k 字符的长 Prompt
5   ↓
6 调用 Qwen 生成/辅助生成 task8_operator_manual.md
7   ↓
8 分批调用 Qwen 生成 task8_per_question_analysis.jsonl
9   ↓
10 将 JSONL 渲染为 task8_per_question_analysis.md
```

```
11     ↓
12 人工检查不同 candidate_tag 版本，线上提交评分后选择最终版本
13
14 阶段二：最终代码生成
15 原始任务 JSON
16     ↓
17 读取 task8_operator_manual.md
18     ↓
19 读取 task8_per_question_analysis.jsonl
20     ↓
21 逐题抽取函数名、任务类型、算子链和对应分析
22     ↓
23 构造短 Prompt 调用 Qwen 生成代码
24     ↓
25 清洗模型输出
26     ↓
27 语法与结构校验
28     ↓
29 失败时修复或规则 fallback
30     ↓
31 写入最终 JSONL 提交文件
```

## 4. 中间文件生成过程

### 4.1 为什么要额外构建中间文件

如果直接让 Qwen 对每道测试题生成代码，模型在一次请求中需要同时完成以下任务：

1. 从自然语言题面中识别功能；
2. 从 Wrapper Entry 中抽取函数名；
3. 判断参数顺序、默认值和关键字参数；
4. 判断属于哪类算子任务；
5. 选择 PyTorch API 或 Triton 实现；
6. 处理 `out=`、`inplace`、`dim`、`keepdim`、`dtype`、`eps`、`alpha`、`beta` 等参数；
7. 写出完整 Python 代码；
8. 避免输出 markdown、JSON 或解释文字；
9. 避免生成无法解析的语法；
10. 在复杂任务中避免错误 Triton kernel。

这些要求全部集中到一次生成中，会明显增加失败概率。中间文件的作用是把任务拆开：

- 第一阶段做“理解和总结”；
- 第二阶段做“代码实现”。

这样最终模型在生成代码时，不需要重新长篇分析，只需要按照已经总结好的函数名、family、算子链和实现策略写代码。

## 4.2 build\_task8\_kb\_files\_no\_autoscore.py 的设计原则

中间知识生成脚本的设计原则如下：

1. 至少构造并使用一次长度大于 20k 字符的 Prompt。
2. 使用 Qwen/OpenAI-compatible API 生成 operator manual 和逐题结构化分析。
3. 不把 hidden chain-of-thought 写入文件。
4. thinking\_steps 只保存公开可展示的简化解题步骤摘要。
5. 不直接生成最终提交答案，只生成中间知识文件。

脚本开头的注释已经明确说明：

```
1 """
2 用途：
3     为 OpenSeek-8 kernel generation 任务生成三个中间知识文件：
4     1) task8_operator_manual.md
5     2) task8_per_question_analysis.jsonl
6     3) task8_per_question_analysis.md
7
8 设计原则：
9     - 至少构造并使用一次长度大于 20k 字符的 Prompt，以满足题目对长上下文使用的要求。
10    - 使用 Qwen/OpenAI-compatible API 生成 operator manual 和逐题结构化分析。
11    - 不进行“多版本自动评分”。线上平台分数无法由本地脚本获得，因此不同版本仅按 candidate_tag 保
    存，
12    最终版本由人工提交到线上平台后根据真实分数选择。
13    - thinking_steps 字段只保存可公开展示的简化解题步骤摘要，不要求模型输出隐藏思维链。
14 """
```

## 4.3 构造一次大于 20k 字符的长 Prompt

题目要求中需要体现一次大于 20k 的 Prompt 使用。因此，中间文件生成脚本专门提供 build\_long\_kb\_prompt 函数，用于构造长上下文知识构建 Prompt。

核心代码如下：

```
1 def build_long_kb_prompt(
2     task_id: str,
3     definitions: List[str],
4     examples: List[Dict[str, Any]],
5     test_samples: List[Dict[str, Any]],
6     min_chars: int = 20000,
7 ) -> str:
8     family_counter = Counter(detect_task_family(get_sample_input(s)) for s in
    test_samples)
9     header = f"""
10 你是 OpenSeek-8 kernel generation 任务的中间知识构建助手。
11
12 任务 ID: {task_id}
13
14 目标：
```

```

15 1. 从训练样例答案与测试题目描述中总结常见 PyTorch/F API 使用模式。
16 2. 形成一个算子使用手册 task8_operator_manual.md。
17 3. 为每道测试题抽取 wrapper、任务类型、算子链、实现策略和公开的简化解题步骤。
18 4. 输出内容必须偏工程化、可复现、可直接被后续代码生成脚本检索使用。
19 5. thinking_steps 只写可公开展示的简要解题步骤，不输出隐藏思维链。
20
21 任务定义：
22 {json.dumps(definitions, ensure_ascii=False, indent=2)}
23
24 测试集 family 初步统计：
25 {json.dumps(family_counter, ensure_ascii=False, indent=2)}
26
27 生成要求：
28 - 优先 PyTorch fallback，不盲目生成 Triton。
29 - 对 conv/matmul/attention/linalg/indexing 等复杂任务，先保证语义正确与参数兼容。
30 - 记录 out=、inplace、dim、keepdim、dtype、eps、alpha、beta、training、p 等常见参数。
31 - 对每题输出 compact JSON 字段：
    id/function/family/description/wrapper_signature/math/other/detected_ops/answer_apis/
    answer_summary/thinking_steps。
32 """.strip()

```

该 Prompt 的内容不是无意义填充，而是由以下部分组成：

1. 任务 ID；
2. 任务定义；
3. 测试集 family 初步统计；
4. 训练样例输入和参考答案代码摘要；
5. 测试样例输入片段；
6. 全局生成策略；
7. 参数处理要求；
8. 输出字段要求；
9. PyTorch fallback 策略；
10. 避免复杂 Triton 的限制。

为了保证长度满足要求，脚本会不断追加训练样例和测试样例摘要；如果数据本身不足，还会追加重复但有用的工程规则：

```

1 filler = "\n".join([
2     "请继续保持：函数名必须完全一致；优先 PyTorch fallback；避免复杂 Triton；输出字段稳定；
  thinking_steps 为公开简化步骤。"
3     for _ in range(200)
4 ])
5 while len(prompt) < min_chars:
6     prompt += "\n" + filler
7
8 assert len(prompt) >= min_chars, f"long prompt is too short: {len(prompt)} <
  {min_chars}"

```

随后脚本会把长 Prompt 写入文件：

```
1 long_prompt_path = out_dir / f"task8_long_prompt_{args.candidate_tag}.txt"
2 long_prompt_path.write_text(long_prompt, encoding="utf-8")
3 print(f"long_prompt_chars={len(long_prompt)}")
```

该文件用于复查生成过程确实使用过一次大于 20k 的 Prompt。

## 4.4 使用长 Prompt 生成 `task8_operator_manual.md`

生成算子手册时，脚本把前面构造好的长 Prompt 作为上下文，再追加手册格式要求：

```
1 def generate_operator_manual_md(client: OpenAI, model: str, long_prompt: str,
2   out_path: Path) -> str:
3     user_prompt = f"""
4     {long_prompt}
5     请基于以上长上下文，生成 task8_operator_manual.md。
6
7     格式要求：
8     # Task8 答案中常用算子使用手册
9
10    本手册根据题目输入与答案代码中出现的 PyTorch/F API 总结，每个算子给出用途、场景和 demo。
11
12    ## 高频算子统计
13
14    然后按照算子逐个说明，每个算子包含：
15    - 用途
16    - 典型场景
17    - Demo 代码块
18
19    只输出 Markdown，不要输出 JSON，不要解释你如何思考。
20    """.strip()
21    print(f"[INFO] operator manual prompt chars = {len(user_prompt)}")
22    assert len(user_prompt) > 20000, "At least one Qwen prompt must be >20k chars."
```

这里的关键点有三个：

1. `operator manual` 的生成 Prompt 明确大于 20k；
2. 输出格式限定为 Markdown；
3. 不要求模型解释内部思考，只要求输出工程文档。

生成后，如果模型输出没有以 Markdown 标题开头，脚本会自动补充标题：

```
1 if not content.startswith("#"):
2     content = "# Task8 答案中常用算子使用手册\n\n" + content
3 out_path.write_text(content, encoding="utf-8")
```

最终得到：



该文件在最终预测脚本中被读取并按 family 选择片段。

## 4.5 分批生成 task8\_per\_question\_analysis.jsonl

逐题分析文件数量较多，如果一次性让模型输出 166 行 JSONL，容易出现截断、格式错误或漏题。因此脚本采用 batch 方式生成。

批次 Prompt 构造函数如下：

```

1  def build_analysis_batch_prompt(
2      definitions: List[str],
3      examples: List[Dict[str, Any]],
4      batch: List[Dict[str, Any]],
5      operator_manual_md: str,
6      batch_start: int,
7  ) -> str:
8      example_blocks = []
9      for i, ex in enumerate(examples[:6], 1):
10         example_blocks.append(compact_example_block(ex, i, max_input_chars=1000,
11             max_output_chars=1200))
12
13     sample_blocks = []
14     for offset, sample in enumerate(batch):
15         sample_blocks.append(compact_test_block(sample, batch_start + offset,
16             max_input_chars=1800))
17
18     prompt = f"""
19     请为下面这一批 OpenSeek-8 测试题生成结构化分析 JSONL。
20
21     任务定义：
22     {json.dumps(definitions, ensure_ascii=False, indent=2)}
23
24     算子手册摘要：
25     {operator_manual_md[:5000]}
26
27     参考训练样例：
28     {"\n\n".join(example_blocks)}
29
30     待分析测试题：
31     {"\n\n".join(sample_blocks)}
32
33     输出要求：
34     - 只输出 JSONL，每个测试题一行。
35     - 每行必须是合法 JSON 对象。
36     - 字段必须包含：
37       id, function, family, description, wrapper_signature, math, other,
38       detected_ops, answer_apis, answer_summary, thinking_steps
39     - detected_ops 和 answer_apis 用字符串数组。
40     - thinking_steps 用字符串数组，写 3 到 5 条可公开展示的简化解题步骤，不输出隐藏思维链。

```

```

39 - family 从以下集合中选择: conv_norm_pool, matmul_linear, attention_softmax_loss,
    linalg, indexing, activation, reduction, quantization, elementwise_math, generic。
40 """.strip()
41     return prompt

```

该 Prompt 明确要求：

- 每行一个 JSON 对象；
- 字段必须完整；
- `detected_ops` 和 `answer_apis` 必须是数组；
- `thinking_steps` 是公开简化步骤，不是隐藏思维链；
- `family` 必须从固定集合中选择。

## 4.6 JSONL 输出解析与补全

模型输出 JSONL 时，可能出现单行 JSON 不合法、字段缺失或 ID 不一致等问题。因此脚本需要进行解析和补全。核心原则是：

1. 保留原始 sample id，因为最终线上提交依赖这个 ID；
2. 如果 Qwen 输出缺字段，则用规则抽取补齐；
3. 如果 family 缺失，则用本地规则 `detect_task_family` 重新判断；
4. 如果 function 缺失，则用 `extract_function_name` 从题目输入中抽取；
5. 如果 detected\_ops 缺失，则用 `extract_ops` 补充；
6. 如果 thinking\_steps 缺失，则用模板步骤补充。

这保证了 `task8_per_question_analysis.jsonl` 即使在模型输出不稳定时，也能保持基本可用。

## 4.7 渲染生成 `task8_per_question_analysis.md`

当 JSONL 生成完成后，脚本会调用 `render_analysis_md` 生成 Markdown 可读版本。Markdown 文件不是主脚本直接读取的输入，但方便人工检查。例如每道题会展示：

```

1  ## 1. openseek-8-xxx - `fused_bmm_rmsnorm_gelu_dropout_sub`
2
3  - **任务类型**: matmul_linear
4  - **Wrapper**: ...
5  - **功能描述**: ...
6  - **数学定义**: ...
7  - **题目算子链**: ...
8  - **答案中显式 API**: ...
9  - **答案实现风格**: ...
10 - **拆解思路**:
11   1. 先识别 wrapper 函数名。
12   2. 映射到 PyTorch 算子链。
13   3. 优先使用 PyTorch fallback。
14   4. 处理 out= 和参数兼容。

```

该文件的价值在于：它可以让人工快速发现异常。例如：

- wrapper 函数名是否错误；
- family 是否明显不合理；
- detected\_ops 是否与题面不符；
- thinking\_steps 是否过空；
- 是否漏掉某些题目。

## 4.8 版本记录与人工线上评分

脚本会生成 `manifest.json`，记录当前候选版本信息。关键字段如下：

```
1 manifest = {
2     "candidate_tag": candidate_tag,
3     "created_at": datetime.now().isoformat(timespec="seconds"),
4     "model": model,
5     "task_id": task_id,
6     "long_prompt_chars": long_prompt_chars,
7     "long_prompt_requirement": "satisfied" if long_prompt_chars > 20000 else
    "not_satisfied",
8     "analysis_items": len(rows),
9     "family_distribution": dict(family_counter),
10    "files": files,
11    "scoring": {
12        "auto_score": False,
13        "reason": "线上评测分数只能通过提交平台获得，本脚本不做多版本自动评分。",
14        "selection_method": "人工将不同 candidate_tag 版本接入主生成脚本并提交线上评测，根据
    真实分数选择最终版本。",
15    },
16 }
```

## 4.9 中间文件构建命令示例

实际运行中间文件构建脚本时，可以使用如下命令：

```
1 export OPENAI_API_KEY=dummy
2 export OPENAI_BASE_URL=http://127.0.0.1:8000/v1
3
4 python build_task8_kb_files_no_autoscore.py \
5     --data /Users/ks/Desktop/LongContext-ICL-Annotation/data/opensseek-
    8_kernel_generation.json \
6     --out-dir /Users/ks/Desktop/LongContext-ICL-Annotation/kb_candidates/v4 \
7     --candidate-tag v4 \
8     --model /Qwen3-4B/Qwen/Qwen3-4B \
9     --batch-size 8
```

输出目录中会包含：

```
1 task8_long_prompt_v4.txt
2 task8_operator_manual.md
3 task8_per_question_analysis.jsonl
4 task8_per_question_analysis.md
5 manifest.json
```

其中：

- `task8_long_prompt_v4.txt` 用于记录大于 20k 的长 Prompt；
- `task8_operator_manual.md` 是生成的算子手册；
- `task8_per_question_analysis.jsonl` 是主脚本需要读取的逐题分析；
- `task8_per_question_analysis.md` 是人工检查版本；
- `manifest.json` 记录版本信息和人工线上评分说明。

## 5. 主预测脚本实现细节

### 5.1 数据加载

主脚本通过 `task_data_loader` 加载原始任务数据：

```
1 def task_data_loader(file_path: str) -> Tuple[str, List[Dict[str, Any]],
2   List[Dict[str, Any]], List[str]]:
3     with open(file_path, "r", encoding="utf-8") as f:
4         data = json.load(f)
5
6     return (
7         data.get("task_id", ""),
8         data.get("examples", []),
9         data.get("test_samples", []),
10        data.get("Definition", []),
11    )
```

该函数返回四类信息：

1. `task_id`：任务 ID；
2. `examples`：训练样例；
3. `test_samples`：待预测样例；
4. `definitions`：任务定义。

其中，训练样例主要用于清洗和潜在上下文支持；测试样例是最终需要生成代码的对象。

### 5.2 中间文件读取

主脚本读取 `task8_operator_manual.md`：

```

1 def load_text_file(path: str, default: str = "") -> str:
2     if not path or not os.path.exists(path):
3         print(f"[WARN] text file not found: {path}")
4         return default
5     with open(path, "r", encoding="utf-8") as f:
6         return f.read().strip()

```

读取逐题分析 JSONL:

```

1 def load_question_analysis(path: str) -> Dict[str, Dict[str, Any]]:
2     results: Dict[str, Dict[str, Any]] = {}
3
4     if not path or not os.path.exists(path):
5         print(f"[WARN] question analysis file not found: {path}")
6         return results
7
8     with open(path, "r", encoding="utf-8") as f:
9         for line_no, line in enumerate(f, 1):
10            line = line.strip()
11            if not line:
12                continue
13            try:
14                obj = json.loads(line)
15            except json.JSONDecodeError:
16                print(f"[WARN] skip invalid analysis JSONL line {line_no}")
17                continue
18
19            sid = obj.get("id") or obj.get("test_sample_id") or obj.get("sample_id")
20            if sid:
21                results[str(sid)] = obj
22
23     return results

```

这里将 JSONL 转换为字典，key 是 sample id，value 是该题的分析对象。这样最终生成每道题时可以 O(1) 查到当前样例对应的分析。

## 5.3 断点续跑机制

代码生成任务容易耗时较长。如果中途失败，不应该重新生成所有样例。因此脚本实现了断点续跑：

```

1 def load_existing_predictions(out_path: str) -> Dict[str, Dict[str, Any]]:
2     existing: Dict[str, Dict[str, Any]] = {}
3
4     if not os.path.exists(out_path):
5         return existing
6
7     with open(out_path, "r", encoding="utf-8") as f:
8         for line_no, line in enumerate(f, 1):
9             line = line.strip()
10            if not line:

```

```

11         continue
12
13     try:
14         obj = json.loads(line)
15     except json.JSONDecodeError:
16         print(f"[WARN] skip invalid JSONL output line {line_no}")
17         continue
18
19     sid = obj.get("test_sample_id") or obj.get("id")
20     pred = obj.get("prediction", "")
21
22     has_prediction = False
23     if isinstance(pred, str):
24         has_prediction = bool(pred.strip())
25     elif isinstance(pred, list):
26         has_prediction = bool(pred and str(pred[0]).strip())
27
28     if sid and has_prediction:
29         existing[str(sid)] = obj
30
31     return existing

```

该函数不仅兼容 `prediction` 为字符串的情况，也兼容旧版本中 `prediction` 为 list 的情况。这样可以避免因为格式变化导致已完成样例被重复生成。

## 5.4 输出 JSONL 写入

最终每道题的结果会通过 `safe_jsonl_row` 统一封装：

```

1 def safe_jsonl_row(test_sample_id: str, code: str) -> Dict[str, Any]:
2     return {"test_sample_id": test_sample_id, "prediction": code}

```

再通过 `append_jsonl` 追加到输出文件：

```

1 def append_jsonl(out_path: str, row: Dict[str, Any]) -> None:
2     out_dir = os.path.dirname(out_path)
3     if out_dir:
4         os.makedirs(out_dir, exist_ok=True)
5
6     with open(out_path, "a", encoding="utf-8") as f:
7         f.write(json.dumps(row, ensure_ascii=False) + "\n")

```

这种写法支持边生成边保存，避免全部样例生成完后才写文件造成中途结果丢失。

## 6. 题目信息抽取与任务分类

### 6.1 文本标准化

主脚本首先对输入文本进行标准化：

```

1 def normalize_text(text: Optional[str]) -> str:
2     if text is None:
3         return ""
4
5     text = str(text)
6     text = text.replace("\r\n", "\n").replace("\r", "\n")
7     text = re.sub(r"[ \t]+", " ", text)
8     text = re.sub(r"\n{3,}", "\n\n", text)
9     return text.strip()

```

该步骤可以统一换行符、压缩多余空格和空行，减少正则抽取时的不稳定性。

## 6.2 抽取 Wrapper Entry

题目中通常包含 `Wrapper Entry Information`，主脚本通过正则提取其中的函数签名：

```

1 def extract_wrapper_entry(input_text: str) -> str:
2     text = normalize_text(input_text)
3
4     m = re.search(
5         r"Wrapper Entry Information:\s*(.*?)(?:\n\s*Args:|\n\s*Keyword
6         args:|\n\s*Returns:|\n\s*Math:|\Z)",
7         text,
8         flags=re.DOTALL | re.IGNORECASE,
9     )
10    return m.group(1).strip() if m else ""

```

该函数的作用是从原始题面中截取 wrapper 信息，为后续函数名抽取和 Prompt 构造提供依据。

## 6.3 抽取函数名

函数名是否正确是线上评测的关键。如果生成代码没有定义题目要求的同名函数，即使内部逻辑正确，也无法通过调用。因此主脚本专门提供 `extract_function_name`：

```

1 def extract_function_name(input_text: str) -> str:
2     text = normalize_text(input_text)
3     entry = extract_wrapper_entry(text)
4
5     m = re.search(r"(?:def\s+)?([A-Za-z_]\w*)\s*\(", entry)
6     if m:
7         return m.group(1)
8
9     m = re.search(
10        r"(?:function|wrapper|entry)\s+[\`'\"]?([A-Za-z_]\w*)[\`'\"]?",
11        text,
12        flags=re.IGNORECASE,
13    )
14    if m:
15        return m.group(1)
16

```

```

17     lower = text.lower()
18     if "mean value" in lower or "computes the mean" in lower:
19         return "mean"
20     if "square system of linear equations" in lower:
21         return "solve"
22     if "conv2d" in lower and "add" in lower:
23         return "conv2d_add"
24
25     return "generated_function"

```

其逻辑为：

1. 优先从 Wrapper Entry 中抽取 `xxx(...)` 形式；
2. 如果失败，则从正文中寻找 function/wrapper/entry 后面的名称；
3. 对少数常见描述进行规则兜底；
4. 最后返回 `generated_function` 作为保底名称。

## 6.4 抽取 wrapper 签名

为了在 Prompt 中保留原始函数签名，主脚本使用：

```

1 def extract_wrapper_signature(input_text: str) -> str:
2     entry = extract_wrapper_entry(input_text)
3     if entry:
4         return entry.splitlines()[0].strip()
5     return ""

```

原始签名能提醒模型参数顺序和默认值，尤其对 `out=None`、`training=True`、`dim=-1` 等参数非常重要。

## 6.5 任务 family 分类

Task8 题目类型复杂，主脚本通过关键词将题目分为多个 family：

```

1 def detect_task_family(input_text: str) -> str:
2     lower = input_text.lower()
3
4     if any(x in lower for x in [
5         "conv2d", "conv1d", "conv3d", "pool2d", "batch_norm",
6         "instance_norm", "layer_norm", "group_norm", "pixel_shuffle",
7         "adaptive_avg_pool2d", "max_pool2d", "avg_pool2d",
8     ]):
9         return "conv_norm_pool"
10
11     if any(x in lower for x in [
12         "linear", "matmul", "matrix multiplication", "mm(", " bmm",
13         "torch.bmm", "mv", "addmm", "einsum", "matrix-vector", "matrix vector",
14     ]):
15         return "matmul_linear"
16
17     if any(x in lower for x in [

```



```

18         "attention", "softmax", "log_softmax", "cross_entropy",
19         "dropout", "transformer", "scaled dot-product",
20     ]):
21         return "attention_softmax_loss"
22
23     if any(x in lower for x in [
24         "svd", "qr", "lu", "cholesky", "solve", "inverse", "invert",
25         "determinant", "det(", "eigen", "eig", "pinv", "lstsq",
26         "least squares", "matrix_power", "matrix power",
27     ]):
28         return "linalg"

```

分类结果用于两件事：

1. 从 operator manual 中选择相关片段；
2. 决定生成策略，例如复杂 family 优先使用 PyTorch fallback。

## 6.6 抽取算子链

主脚本通过 `extract_ops` 将题目关键词映射为 PyTorch API：

```

1  def extract_ops(input_text: str) -> List[str]:
2      lower = input_text.lower()
3      ops: List[str] = []
4
5      candidates = [
6          ("conv2d", "F.conv2d"),
7          ("linear", "F.linear"),
8          ("bmm", "torch.bmm"),
9          ("matmul", "torch.matmul"),
10         ("batch_norm", "F.batch_norm"),
11         ("softmax", "F.softmax"),
12         ("dropout", "F.dropout"),
13         ("relu", "F.relu"),
14         ("gelu", "F.gelu"),
15         ("sigmoid", "torch.sigmoid"),
16         ("sqrt", "torch.sqrt"),
17         ("mean", "torch.mean"),
18         ("sum", "torch.sum"),
19         ("gather", "torch.gather"),
20         ("svd", "torch.linalg.svd"),
21         ("qr", "torch.linalg.qr"),
22         ("solve", "torch.linalg.solve"),
23     ]

```

实际代码中的候选表更长，覆盖 elementwise、activation、reduction、indexing、linalg、quantization 等多类算子。抽取结果会写入 Prompt 的 `Operation chain` 部分，引导模型按顺序组合 API。

## 6.7 构建生成策略

主脚本通过 family 决定策略：

```

1 def build_strategy(family: str, ops: List[str]) -> str:
2     if family in {
3         "conv_norm_pool", "matmul_linear", "attention_softmax_loss",
4         "linalg", "indexing", "activation", "reduction",
5     }:
6         return (
7             "Use PyTorch fallback only. Do not use complex Triton. "
8             "Compose the detected operations step by step in the listed order."
9         )
10
11     if family == "quantization":
12         return (
13             "Prefer PyTorch arithmetic implementation. Avoid complex Triton unless
14             the operation is simple row-wise dequantization."
15         )
16
17     if family == "elementwise_math":
18         return (
19             "Use PyTorch elementwise operations. Triton is allowed only for a very
20             simple elementwise kernel, but PyTorch fallback is preferred."
21         )
22
23     return "Use the safest PyTorch fallback implementation. Do not generate complex
24     Triton."

```

这里的核心是“先正确，再性能”。对复杂题目而言，PyTorch fallback 通常更容易保证语义和参数兼容，避免模型生成错误 Triton。

## 7. Prompt-KB 选择与最终 Prompt 构造

### 7.1 截断中间文本

主脚本并不会把完整 operator manual 和完整 per-question analysis 全部放进 Prompt。为了避免 Prompt 过长导致代码生成被截断，脚本设置：

```

1 MAX_OPERATOR_MANUAL_CHARS = 2500
2 MAX_QUESTION_ANALYSIS_CHARS = 1200

```

并通过 `truncate_text` 截断文本：

```

1 def truncate_text(text: str, max_chars: int) -> str:
2     text = normalize_text(text)
3     if len(text) <= max_chars:
4         return text
5     return text[:max_chars] + "\n\n...[TRUNCATED]..."

```

这说明：

- `>20k Prompt` 只用于中间文件构建阶段；
- 最终逐题代码生成阶段使用的是短 Prompt；
- 这样可以降低模型输出代码被截断的风险。

## 7.2 按 family 选择 operator manual 片段

主脚本根据当前题目的 family，从算子手册中筛选相关行：

```
1 def select_operator_manual_section(operator_manual: str, family: str, max_chars: int
  = MAX_OPERATOR_MANUAL_CHARS) -> str:
2     manual = normalize_text(operator_manual)
3     if not manual:
4         return ""
5
6     keywords_by_family = {
7         "activation": ["ReLU", "Sqrt", "Exp", "Log", "Sigmoid", "Tanh", "GELU",
  "Softplus"],
8         "elementwise_math": ["Sqrt", "Exp", "Log", "Sigmoid", "Tanh", "zeta",
  "chebyshev", "rsqrt"],
9         "reduction": ["Reduction", "max", "logsumexp", "rsqrt", "sum_std", "std",
  "mean", "norm"],
10        "indexing": ["Index", "Index Fill", "Index Select", "gather", "masked", "赋
  值"],
11        "linalg": ["solve", "svd", "qr", "cholesky", "det", "inverse", "linalg"],
12        "matmul_linear": ["linear", "bmm", "matmul", "mv", "matrix",
  "softplus_linear", "fused_mv"],
13        "conv_norm_pool": ["conv", "conv2d", "batch_norm", "pool", "normalization"],
14        "attention_softmax_loss": ["attention", "softmax", "cross_entropy",
  "dropout"],
15        "quantization": ["quantize", "dequantize", "int8", "fp8"],
16    }
```

该函数的优点是：

1. 当前题只读取相关算子知识；
2. 避免 Prompt 过长；
3. 减少无关 API 干扰；
4. 保持不同 family 的生成策略差异。

## 7.3 选择逐题分析

主脚本通过 sample id 获取当前题分析：

```

1 def select_question_analysis(sample_id: str, question_analysis: Dict[str, Dict[str,
  Any]]) -> str:
2     obj = question_analysis.get(sample_id, {})
3     if not obj:
4         return "No precomputed analysis for this sample."
5     text = json.dumps(obj, ensure_ascii=False, indent=2)
6     return truncate_text(text, MAX_QUESTION_ANALYSIS_CHARS)

```

如果某道题在 JSONL 中没有找到分析，也不会直接崩溃，而是返回提示文本。最终 Prompt 仍然可以依赖本地抽取规则生成。

## 7.4 固定 helper 代码

为了统一处理 `out=` 写回，Prompt 中固定包含 `_write_out`：

```

1 FIXED_HELPER_CODE = """def _write_out(value, out=None):
2     if out is None:
3         return value
4     if isinstance(value, tuple):
5         if isinstance(out, tuple):
6             for dst, src in zip(out, value):
7                 dst.copy_(src)
8             return out
9         return value
10    out.copy_(value)
11    return out
12 """

```

该 helper 解决三类问题：

1. `out is None` 时直接返回结果；
2. 如果输出是 tuple 且 out 也是 tuple，则逐项写回；
3. 如果输出是普通 Tensor，则执行 `out.copy_(value)`。

这对 PyTorch API 的兼容性非常重要。

## 7.5 System Prompt

主脚本的 system prompt 明确要求模型输出完整可执行 Python 代码，并把正确性放在最高优先级：

```

1 You are a code-generation model for OpenSeek-8 kernel generation.
2
3 Generate one complete executable Python code answer.
4
5 Priority:
6 1. Correctness
7 2. Importability / executable code
8 3. Exact wrapper function name
9 4. Robust positional and keyword argument handling
10 5. PyTorch semantic equivalence

```

```

11 6. Performance
12
13 Core strategy:
14 - Prefer PyTorch fallback over complex Triton.
15 - Do not blindly write Triton.
16 - Use Triton only for very simple elementwise kernels.
17 - For conv, matmul/bmm, attention, dropout, normalization, indexing, linalg, and
    complex fused blocks, use PyTorch.

```

这部分约束了模型的整体行为，避免其为了性能盲目写 Triton。

## 7.6 User Prompt 模板

最终每道题的 user prompt 包含四部分：

```

1 A. Relevant operator manual
2 B. Extracted rules for current sample
3 C. Current original test sample
4 D. Required output

```

模板中的关键字段包括：

```

1 USER_PROMPT_TEMPLATE = """
2 Solve this OpenSeek-8 test sample.
3
4 A. Relevant operator manual:
5 {operator_manual_for_sample}
6
7 B. Extracted rules for current sample:
8 Sample ID: {sample_id}
9 Wrapper function name: {function_name}
10 Original wrapper signature: {wrapper_signature}
11 Task family: {family}
12 Operation chain:
13 {ops_text}
14 Strategy: {strategy}
15
16 Precomputed analysis:
17 {analysis_text}
18
19 C. Current original test sample:
20 {current_input}
21
22 D. Required output:
23 Generate ONLY executable Python code.
24
25 The code MUST:
26 - start with import torch
27 - include import torch.nn.functional as F when useful
28 - include this exact helper code:
29

```

```

30 {fixed_helper_code}
31
32 - define exactly:
33   def {function_name}(*args, **kwargs):
34 - support positional args and kwargs
35 - support out= if present
36 - compose PyTorch operations step by step
37 - avoid complex Triton
38 - not be a minimal one-line wrapper
39 - not output explanation or markdown
40 - never write assignment expressions inside function call arguments
41 - use separate assignment lines before function calls
42 - never write code like: func(arg = value = other)
43
44 Final answer:
45 """.strip()

```

这说明最终 Prompt 并不是简单地把题目丢给模型，而是把以下信息全部显式提供给模型：

- 当前题 ID；
- 目标函数名；
- 原始 wrapper 签名；
- 任务 family；
- 算子链；
- 实现策略；
- 预生成逐题分析；
- 原始题面；
- 输出格式要求；
- 必须包含的 helper 代码。

## 7.7 构造 messages

最终由 `build_messages` 汇总上述信息：

```

1  def build_messages(
2      sample: Dict[str, Any],
3      definitions: List[str],
4      examples: List[Dict[str, Any]],
5      operator_manual: str,
6      question_analysis: Dict[str, Dict[str, Any]],
7  ) -> List[Dict[str, str]]:
8      current_input = normalize_text(sample.get("input", ""))
9      sample_id = str(sample.get("id", ""))
10
11     function_name = extract_function_name(current_input)
12     wrapper_signature = extract_wrapper_signature(current_input)
13     family = detect_task_family(current_input)

```

```

14 ops = extract_ops(current_input)
15 strategy = build_strategy(family, ops)
16 ops_text = build_ops_text(ops)
17
18 operator_manual_for_sample = select_operator_manual_section(operator_manual,
family)
19 analysis_text = select_question_analysis(sample_id, question_analysis)

```

该函数体现了 Prompt-KB 的核心：

```

1 当前题输入
2   ↓
3 本地规则抽取 function/family/ops
4   ↓
5 从 operator manual 选相关知识
6   ↓
7 从 JSONL 取当前题分析
8   ↓
9 拼成最终 messages

```

## 8. 模型调用、清洗与验证

### 8.1 Qwen API 调用

主脚本通过 OpenAI-compatible 接口调用 Qwen：

```

1 client = OpenAI(
2     api_key=os.getenv("OPENAI_API_KEY", "dummy"),
3     base_url=os.getenv(
4         "OPENAI_BASE_URL",
5         "https://flagos.io/flagos-lab/hw/node/HW-gpu57/port/22653/v1",
6     ),
7 )

```

调用函数为：

```

1 def qwen_api(
2     messages: List[Dict[str, str]],
3     model: str = MODEL_NAME,
4     retries: int = 3,
5     sleep_base: float = 6.0,
6 ) -> str:
7     for attempt in range(retries):
8         try:
9             res = client.chat.completions.create(
10                 model=model,
11                 messages=messages,
12                 temperature=0.0,
13             )

```

```

14         return res.choices[0].message.content or ""
15     except Exception as e:
16         if attempt == retries - 1:
17             print(f"\n[ERROR] API call failed after {retries} attempts: {e}")
18             return ""
19         wait = sleep_base * (2 ** attempt) + random.random()
20         time.sleep(wait)
21     return ""

```

这里使用 `temperature=0.0`，目的是降低随机性，让输出更稳定。重试机制可以缓解网络或服务临时错误。

## 8.2 清洗模型输出

模型有时会输出 markdown 代码块、解释文字或额外格式。主脚本通过 `clean_code_response` 提取纯代码：

```

1  def clean_code_response(text: str) -> str:
2      if not text:
3          return ""
4
5      text = str(text).strip()
6
7      fence = re.search(
8          r"```(?:python|py)?\s*(.*?)```",
9          text,
10         flags=re.DOTALL | re.IGNORECASE,
11     )
12     if fence:
13         text = fence.group(1).strip()
14
15     start_markers = ["import ", "from ", "@triton", "@torch", "def ", "class "]
16     positions = [text.find(m) for m in start_markers if text.find(m) != -1]
17     if positions:
18         text = text[min(positions):].strip()
19
20     return text.replace("```", "").strip()

```

该函数处理三类问题：

1. 移除 ```python 代码块；
2. 如果前面有解释文字，则从第一个 import/def/class 等位置开始截取；
3. 删除残留的反引号。

## 8.3 基础代码校验

清洗后的代码会进入 `validate_code_basic`：

```

1  def validate_code_basic(code: str, expected_func_name: str, family: str) ->
    Tuple[bool, str]:
2      if not code or not code.strip():

```



```

3         return False, "empty output"
4
5     if "`" in code:
6         return False, "markdown fence remains"
7
8     if re.search(r"\bTODO\b|\.\.\.", code):
9         return False, "placeholder detected"
10
11    if re.search(r"def\s+\w+\s*\([^)]*\):\s*\n\s*pass\s*(?:\n|$)", code):
12        return False, "pass-only function detected"
13
14    try:
15        tree = ast.parse(code)
16    except SyntaxError as e:
17        return False, f"syntax error: {e}"

```

校验项目包括：

- 输出是否为空；
- 是否残留 markdown；
- 是否包含 TODO 或省略号；
- 是否是 pass-only 函数；
- 是否能被 Python AST 解析。

随后检查函数名是否存在：

```

1 func_names = {
2     node.name for node in ast.walk(tree)
3     if isinstance(node, (ast.FunctionDef, ast.AsyncFunctionDef))
4 }
5 class_names = {
6     node.name for node in ast.walk(tree)
7     if isinstance(node, ast.ClassDef)
8 }
9
10 if expected_func_name not in func_names and expected_func_name not in class_names:
11     return False, f"expected wrapper `{expected_func_name}` not defined"

```

这可以防止模型定义错函数名。

## 8.4 检测可疑 Triton

复杂 Triton 是本任务常见失败来源。主脚本通过 `has_complex_or_suspicious_triton` 拦截部分高风险输出：

```

1 def has_complex_or_suspicious_triton(code: str, family: str) -> bool:
2     if "@triton.jit" not in code and "triton.language" not in code:
3         return False
4
5     complex_families = {
6         "conv_norm_pool",
7         "matmul_linear",
8         "attention_softmax_loss",
9         "linalg",
10        "indexing",
11    }
12    if family in complex_families:
13        return True

```

对于 `conv_norm_pool`、`matmul_linear`、`attention_softmax_loss`、`linalg`、`indexing` 等复杂 family，如果模型生成 Triton，直接视为高风险。这样做的原因是这些任务通常涉及复杂维度、广播、stride 或矩阵分解，Triton 容易产生不可运行代码。

## 8.5 长度检查

对于复杂 family，代码过短通常意味着模型没有真正实现题目逻辑：

```

1 length_sensitive_families = {
2     "conv_norm_pool",
3     "matmul_linear",
4     "attention_softmax_loss",
5     "indexing",
6 }
7
8 if family in length_sensitive_families and len(code) < 500:
9     return False, f"code too short for complex family `{family}`"

```

这可以拦截“一行调用但没有处理参数”的弱实现。

## 9. 修复机制与规则回退

### 9.1 fatal syntax 直接 fallback

主脚本认为语法错误是高风险错误：

```

1 def is_fatal_truncated_syntax(reason: str) -> bool:
2     """
3     Treat every SyntaxError from the model as fatal and use rule fallback directly.
4     For Qwen-4B, repairing syntax errors often wastes time and may produce another
5     malformed answer.
6     """
7     return "syntax error" in reason.lower()

```

如果模型输出存在语法错误，脚本不一定继续修复，而是优先使用规则 fallback。这是因为语法错误往往来自代码截断、括号未闭合或模型生成格式混乱，继续让模型修复可能浪费时间且仍然失败。

## 9.2 repair prompt

如果不是 fatal syntax，且 `ENABLE_REPAIR=True`，脚本会构造修复 Prompt：

```
1  repair_prompt = f"""The previous code was invalid.
2
3  Validation error:
4  {reason}
5
6  Expected wrapper function:
7  {func_name}
8
9  Detected family:
10 {family}
11
12 Detected operations:
13 {build_ops_text(ops)}
14
15 Previous code:
16 {bad_code[:3000]}
17
18 Regenerate corrected code.
19
20 Rules:
21 - Output ONLY Python code.
22 - Define def {func_name}(*args, **kwargs):
23 - Include import torch and import torch.nn.functional as F.
24 - Include the exact _write_out helper from the prompt.
25 - Use PyTorch fallback style.
26 - Do not write complex Triton.
27 - Never write assignment expressions inside function call arguments.
28 - Use separate assignment lines before function calls.
29 - No markdown.
30 """
```

该 Prompt 会告诉模型：

- 上一次错误原因是什么；
- 正确函数名是什么；
- 当前 family 和算子链是什么；
- 上一次代码片段是什么；
- 修复后的输出仍必须是纯 Python 代码。

## 9.3 规则 fallback 模板

当模型生成失败或修复失败时，脚本使用规则模板生成兜底实现：

```

1  FALLBACK_TEMPLATE = """
2  import torch
3  import torch.nn.functional as F
4
5  def _write_out(value, out=None):
6      if out is None:
7          return value
8      if isinstance(value, tuple):
9          if isinstance(out, tuple):
10             for dst, src in zip(out, value):
11                 dst.copy_(src)
12             return out
13         return value
14     out.copy_(value)
15     return out
16
17 def _rms_norm(x, normalized_shape=None, eps=1e-5, weight=None):
18     if normalized_shape is None:
19         dims = (-1,)
20     elif isinstance(normalized_shape, int):
21         dims = (-1,)
22     else:
23         dims = tuple(range(x.dim() - len(tuple(normalized_shape)), x.dim()))
24     y = x * torch.rsqrt(torch.mean(x * x, dim=dims, keepdim=True) + eps)
25     if weight is not None:
26         y = y * weight
27     return y
28
29 def {func_name}(*args, **kwargs):
30     out = kwargs.pop("out", None)
31     name = "{func_name}"

```

fallback 的基本逻辑是：

1. 如果函数名正好是 `torch`、`torch.linalg`、`torch.special` 或 `F` 中已有 API，则直接分发调用；
2. 如果是常见复合函数名，则按规则组合 PyTorch API；
3. 支持 `out=` 写回；
4. 对 RMSNorm、conv2d 组合、linear 激活组合、reduction 组合、solve/cholesky solve 等常见模式提供兜底实现。

例如 conv2d 类 fallback：

```

1  if "conv2d" in name:
2      input = kwargs.get("input", args[0] if len(args) > 0 else None)
3      weight = kwargs.get("weight", args[1] if len(args) > 1 else None)
4      bias = kwargs.get("bias", args[2] if len(args) > 2 else None)
5      stride = kwargs.get("stride", 1)
6      padding = kwargs.get("padding", 0)
7      dilation = kwargs.get("dilation", 1)
8      groups = kwargs.get("groups", 1)

```

```

9     y = F.conv2d(input, weight, bias, stride, padding, dilation, groups)
10    if "add" in name:
11        other = kwargs.get("other", args[3] if len(args) > 3 else None)
12        if other is not None:
13            y = y + kwargs.get("alpha", 1) * other
14    if "batch_norm" in name:
15        running_mean = kwargs.get("running_mean", None)
16        running_var = kwargs.get("running_var", None)
17        bn_weight = kwargs.get("bn_weight", kwargs.get("weight_bn", None))
18        bn_bias = kwargs.get("bn_bias", kwargs.get("bias_bn", None))
19        training = kwargs.get("training", True)
20        momentum = kwargs.get("momentum", 0.1)
21        eps = kwargs.get("eps", 1e-5)
22        y = F.batch_norm(y, running_mean, running_var, bn_weight, bn_bias, training,
momentum, eps)

```

该规则不能保证覆盖全部题目，但能显著降低空输出、语法错误或完全不可运行代码的比例。

## 9.4 单题预测流程

主函数 `predict_one` 将模型生成、校验、修复和 fallback 串联起来：

```

1  def predict_one(...):
2      sid = str(sample.get("id", ""))
3      expected_func_name = extract_function_name(sample.get("input", ""))
4      family = detect_task_family(sample.get("input", ""))
5
6      code = ""
7
8      if use_model:
9          messages = build_messages(...)
10         raw = qwen_api(messages, model=model)
11         code = clean_code_response(raw)
12
13         ok, reason = validate_code_basic(code, expected_func_name, family)
14
15         if not ok and is_fatal_truncated_syntax(reason):
16             if enable_rule_fallback:
17                 return safe_jsonl_row(sid, make_rule_fallback_code(sample))
18
19         if not ok and enable_repair:
20             repaired = repair_code_with_prompt_kb(...)
21             repaired_ok, repaired_reason = validate_code_basic(repaired,
expected_func_name, family)
22             if repaired_ok:
23                 code = repaired
24                 ok, reason = True, "ok after repair"
25             else:
26                 code = repaired
27                 reason = repaired_reason
28

```

```

29         if ok:
30             return safe_jsonl_row(sid, code)
31
32     if enable_rule_fallback:
33         return safe_jsonl_row(sid, make_rule_fallback_code(sample))
34
35     return safe_jsonl_row(sid, code)

```

这一流程保证：

1. 优先使用模型生成结果；
2. 模型结果合法则直接采用；
3. 模型结果非法但可修复则尝试修复；
4. 修复失败则使用规则 fallback；
5. 最终始终尽量输出一个 JSONL row。

## 10. 主程序执行流程

主函数 `main` 的执行步骤如下。

### 10.1 加载数据与中间文件

```

1 def main() -> None:
2     task_id, examples, test_samples, definitions = task_data_loader(FILE_PATH)
3
4     operator_manual = load_text_file(OPERATOR_MANUAL_PATH)
5     question_analysis = load_question_analysis(PER_QUESTION_ANALYSIS_PATH)
6
7     operator_manual = truncate_text(operator_manual, 15000)

```

这里说明主脚本实际读取两个中间文件：

- `task8_operator_manual.md`；
- `task8_per_question_analysis.jsonl`。

`task8_per_question_analysis.md` 不在此处读取，它是人工查看用文件。

### 10.2 清洗训练样例与测试样例

```

1  for ex in examples:
2      ex["input"] = normalize_text(ex.get("input", ""))
3      out = ex.get("output", "")
4      if isinstance(out, list):
5          ex["output"] = [clean_code_response(str(x)) for x in out]
6      else:
7          ex["output"] = [clean_code_response(str(out))]
8
9  for sample in test_samples:
10     sample["input"] = normalize_text(sample.get("input", ""))

```

这一步统一输入格式，并清洗训练样例中的参考代码。

## 10.3 打印运行信息

```

1  print(f"task_id={task_id}")
2  print(f"examples={len(examples)}, test_samples={len(test_samples)}")
3  print(f"system_prompt_chars={len(SYSTEM_PROMPT)}")
4  print(f"operator_manual_chars={len(operator_manual)}")
5  print(f"question_analysis_items={len(question_analysis)}")
6  print(f"model={MODEL_NAME}")
7  print(f"use_model={USE_MODEL}, max_workers={MAX_WORKERS}")
8  print(f"out_path={OUT_PATH}")

```

这些日志可以确认：

- 原始数据是否正确读取；
- 中间文件是否成功加载；
- 模型名称是否正确；
- 输出路径是否正确；
- 当前有多少条逐题分析。

## 10.4 断点续跑和待处理样例

```

1  existing = load_existing_predictions(OUT_PATH)
2  pending_samples = [s for s in test_samples if str(s.get("id", "")) not in existing]
3  print(f"existing predictions={len(existing)}, pending={len(pending_samples)}")
4
5  if not pending_samples:
6      print("All test_samples already generated.")
7      return

```

这样可以避免重复调用模型，节省时间和 API 成本。

## 10.5 多线程生成

```

1  worker = partial(

```

```

2     predict_one,
3     definitions=definitions,
4     examples=examples,
5     operator_manual=operator_manual,
6     question_analysis=question_analysis,
7     model=MODEL_NAME,
8     use_model=USE_MODEL,
9     enable_repair=ENABLE_REPAIR,
10    enable_rule_fallback=ENABLE_RULE_FALLBACK,
11 )
12
13 with ThreadPoolExecutor(max_workers=MAX_WORKERS) as executor:
14     futures = {
15         executor.submit(worker, sample): sample
16         for sample in pending_samples
17     }

```

每个线程处理一道样例。完成后立即写入 JSONL：

```

1  for future in tqdm(as_completed(futures), total=len(futures), desc="Generating"):
2      sample = futures[future]
3      try:
4          result = future.result()
5      except Exception as e:
6          print(f"\n[ERROR] Generation failed: {sample.get('id', '')} | {e}")
7          try:
8              result = safe_jsonl_row(str(sample.get("id", "")),
make_rule_fallback_code(sample))
9          except Exception as fallback_error:
10             print(f"\n[ERROR] Emergency fallback failed: {sample.get('id', '')} |
{fallback_error}")
11             result = safe_jsonl_row(str(sample.get("id", "")), "")
12
13     append_jsonl(OUT_PATH, result)

```

即使某个 future 抛出异常，脚本也会尝试 emergency fallback，避免整个任务中断。

## 11. 三个中间文件的必要性总结

### 11.1 task8\_operator\_manual.md 的必要性

该文件解决的是“全局算子知识复用”问题。Task8 中大量题目都在重复使用相似算子，例如：

- `F.relu`、`F.gelu`、`torch.sigmoid`、`torch.tanh`；
- `torch.sqrt`、`torch.exp`、`torch.log`、`torch.rsqrt`；
- `F.linear`、`torch.bmm`、`torch.matmul`、`torch.mm`、`torch.mv`；
- `F.conv2d`、`F.batch_norm`、`F.layer_norm`、`F.group_norm`；
- `F.softmax`、`F.log_softmax`、`F.dropout`；



- `torch.sum`、`torch.mean`、`torch.std`、`torch.min`、`torch.max`；
- `torch.linalg.solve`、`torch.linalg.svd`、`torch.linalg.qr`、`torch.linalg.cholesky`。

如果每道题都让模型重新回忆这些 API 的使用方式，容易出现参数遗漏。算子手册将这些 API 固化成可检索知识，能提升生成稳定性。

## 11.2 task8\_per\_question\_analysis.jsonl 的必要性

该文件解决的是“单题理解前置化”问题。每道题在最终生成代码前，已经有对应的：

- 函数名；
- family；
- wrapper 签名；
- 功能描述；
- 数学定义；
- 算子链；
- API 候选；
- 实现策略；
- 公开简化步骤。

最终模型不再需要从零分析，而是按照这些字段完成实现。

## 11.3 task8\_per\_question\_analysis.md 的必要性

该文件解决的是“人工可读审查”问题。JSONL 适合程序读取，但不适合人工逐题检查。Markdown 版本可以快速检查 family 分布、题目拆解和实现风格，对发现错误非常有用。

尤其是在没有本地自动线上评分的情况下，人工检查中间文件质量是必要步骤。Markdown 文件可以辅助人工决定哪个 candidate\_tag 更适合提交线上评分。

---

## 12. 版本选择与线上评分说明

由于线上平台分数只能通过提交系统获得，本地脚本不做自动评分。正确流程如下：

1. 用 `build_task8_kb_files_no_autoscore.py` 生成 `candidate_tag=v1` 的中间文件。
2. 将 `v1` 的 `task8_operator_manual.md` 和 `task8_per_question_analysis.jsonl` 接入主脚本。
3. 运行主脚本生成提交 JSONL。
4. 上传线上平台获取真实分数。
5. 修改 Prompt 或中间文件生成策略，生成 `candidate_tag=v2/v3/v4`。
6. 重复提交。
7. 根据线上真实分数选择最终版本。

---

## 13. 稳定性设计

### 13.1 优先保证函数名正确

线上评测通常会导入生成代码并调用指定函数。如果函数名错误，代码逻辑再正确也无法通过。因此主脚本多处强调：

```
1 Define the exact wrapper function name.
2 def {function_name}(*args, **kwargs):
```

同时本地验证会检查 AST 中是否存在目标函数名。

## 13.2 优先 PyTorch fallback

本任务虽然是 kernel generation，但不是所有题都适合写 Triton。对于矩阵分解、线性代数、归一化、attention、indexing 等任务，PyTorch API 更稳定。

因此 system prompt 中明确写道：

```
1 Prefer PyTorch fallback over complex Triton.
2 Do not blindly write Triton.
3 Use Triton only for very simple elementwise kernels.
```

## 13.3 参数兼容

Prompt 中明确要求支持：

```
1 out=, inplace, dim, keepdim, dtype, eps, alpha, beta, threshold, p, dropout_p,
   training, stride, padding, dilation, groups
```

这些参数是 PyTorch wrapper 中最常见的兼容点。漏掉这些参数会导致评测调用失败。

## 13.4 输出格式控制

模型必须只输出 Python 代码：

```
1 Output only Python code.
2 No markdown.
3 No JSON.
4 No explanations.
5 Start with import torch.
```

同时清洗函数会移除 markdown code fence，验证函数会检查是否残留 ````。

## 13.5 修复与 fallback

本方案不是完全相信模型输出，而是形成了多层防线：

```
1 Qwen 生成
2   ↓
3 clean_code_response 清洗
4   ↓
5 validate_code_basic 校验
6   ↓
7 repair_code_with_prompt_kb 修复
8   ↓
9 make_rule_fallback_code 规则回退
```

这样即使模型输出不稳定，也能尽量保证最终 JSONL 有可执行代码。

## 14 本方案最终形成了一条稳定的 Prompt-KB 代码生成流程：

```
1 1. 读取 openseek-8_kernel_generation.json。
2 2. 使用 build_task8_kb_files_no_autoscore.py 构造一次 >20k 长 Prompt。
3 3. 调用 Qwen 生成 task8_operator_manual.md。
4 4. 分批调用 Qwen 生成 task8_per_question_analysis.jsonl。
5 5. 将 JSONL 渲染为 task8_per_question_analysis.md。
6 6. 按 candidate_tag 保存不同候选版本。
7 7. 人工将不同版本接入主脚本并进行线上评分。
8 8. 选择线上表现最好的中间文件版本。
9 9. 运行 task8_prompt_kb_generator copy.py 生成最终提交 JSONL。
10 10. 主脚本逐题读取 operator manual 片段和对应 analysis。
11 11. Qwen 生成代码。
12 12. 本地清洗、校验、修复和 fallback。
13 13. 追加写入最终 JSONL。
```

该流程的关键优势是：

- 将复杂代码生成任务拆成“知识构建”和“代码生成”两个阶段；
- 使用中间文件降低每道题即时分析压力；
- 使用 >20k Prompt 满足长上下文要求；
- 使用人工线上评分保证版本选择真实可信；
- 使用 AST 校验、repair 和 fallback 提升最终输出稳定性；